



Industrial IoT Project

Secure Industrial IoT Monitoring and Data Communication System Using RS485, Protobuf, AES Encryption, and Grafana

Metropolia University of Applied Sciences
Bachelor of Engineering
Smart Automation, Engineering and Technology
Project Report

Date: 09 May 2026

Abstract

Author(s): Bishal Shrestha, Pukar Ale, Pragya Khatiwada

Title: Industrial IoT Project – Secure Industrial IoT Monitoring and Data Communication System

Date: 09 May 2026

Degree: Bachelor of Engineering

Degree Programme: Smart Automation, Engineering and Technology

This Project presents the design and implementation of a secure Industrial Internet of Things (IIoT) monitoring system using industrial communication protocols, edge computing, secure data transmission, and real-time visualization technologies. The project focuses on building a complete IIoT pipeline consisting of sensor nodes, an edge controller, a gateway, and a visualization server.

The implemented system uses RS485 Modbus communication in a daisy-chain sensor configuration connected to an Edge101 Industrial ESP32 controller. Sensor data from a light sensor and ultrasonic sensor are serialized using Protocol Buffers (Protobuf) before transmission over Ethernet to the gateway. The gateway performs Protobuf decoding, AES encryption handling, local database storage, secure logging, firewall filtering, and monitoring operations.

The project also integrates Prometheus and Node Exporter for system monitoring, Grafana for dashboard visualization, SQLite and SQLCipher for local secure databases, WireGuard VPN for secure communication, HiveMQ with TLS encryption for secure MQTT communication, and InfluxDB for time-series data storage.

Additional cybersecurity features include AES-256 encryption, SHA256-based secure logging, firewall implementation using iptables, and secure TLS-based MQTT communication. The implemented architecture demonstrates the practical application of Industrial IoT concepts, industrial protocols, data serialization, cybersecurity, and monitoring systems.

The results show that the developed system successfully achieves secure industrial sensor communication, real-time monitoring, encrypted data transmission, and reliable dashboard visualization suitable for Industrial IoT environments.

Keywords: Industrial IoT, RS485, Modbus, Protobuf, MQTT, Grafana, Prometheus, AES Encryption, SQLCipher, WireGuard

Table of Contents

<i>Introduction.....</i>	<i>5</i>
<i>Industrial Internet of Things Background</i>	<i>5</i>
<i>System Architecture.....</i>	<i>5</i>
<i>Hardware and Software Implementation</i>	<i>6</i>
<i>Cybersecurity Implementation</i>	<i>7</i>
<i>Monitoring and Visualization.....</i>	<i>8</i>
<i>Validation and Testing</i>	<i>8</i>
<i>Results and Discussion</i>	<i>9</i>
<i>Conclusion</i>	<i>11</i>
<i>Appendices.....</i>	<i>13</i>
<i>Appendix 1 – System Pipeline</i>	<i>13</i>
<i>Appendix 2 – Cybersecurity Features</i>	<i>13</i>
<i>Appendix 3 – Monitoring Stack</i>	<i>13</i>
<i>Appendix 4 – Communication Protocols.....</i>	<i>14</i>

Introduction

Industrial Internet of Things (IIoT) refers to the integration of industrial devices, sensors, gateways, communication protocols, and software systems to improve monitoring, automation, and operational efficiency in industrial environments. Compared to traditional IoT systems, IIoT systems require higher reliability, stronger cybersecurity, stable communication, and continuous real-time monitoring, as industrial environments often involve critical infrastructure and long-term operations. This project's objective was to develop and implement a secure Internet of Things (IoT) monitoring system capable of collecting data from industrial sensors, transmitting it, storing it in databases, and displaying the results in real time. The implemented system consists of a light sensor and an ultrasonic sensor, connected to an Edge101 Industrial ESP32 controller via RS485 Modbus communication in a daisy-chain configuration. Before sending encoded payloads to the gateway over Ethernet, the edge controller continuously collects sensor values and serializes them using Protocol Buffers (Protobuf). The gateway receives and decodes the Protobuf messages, stores them locally, monitors system performance, and securely sends the data to the server for visualization. The system included AES encryption, TLS-secured MQTT communication with HiveMQ, SQLCipher-encrypted databases, iptables firewall filtering, SHA256 secure logging, and WireGuard VPN tunneling, among other Industrial IoT technologies and cybersecurity mechanisms. Monitoring and visualization were implemented using Prometheus, Node Exporter, Grafana, and InfluxDB to provide real-time monitoring and dashboard visualization. The project successfully demonstrated the integration of Industrial IoT cybersecurity into a comprehensive IIoT architecture, along with real-time monitoring, secure data transmission, edge computing, and reliable industrial communication.

Industrial Internet of Things Background

To enhance industrial automation and monitoring, industrial Internet of Things systems combine software systems, sensors, industrial communication technologies, and networking. In factories, energy systems, manufacturing plants, and other smart industrial environments where secure communication is essential, IIoT systems are frequently

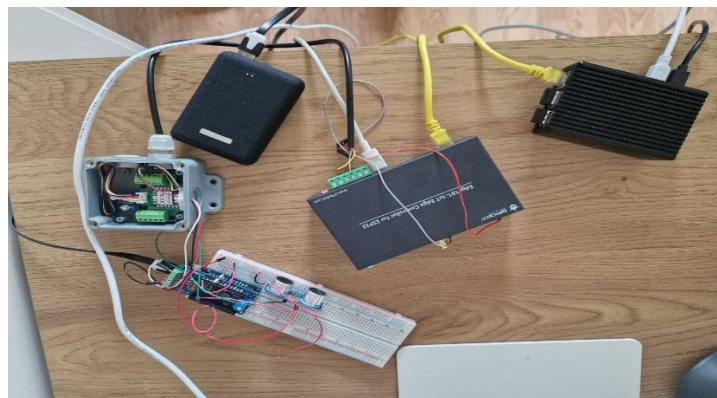
utilized. Unlike normal IoT systems, Industrial IoT environments require long-term operation, high communication stability, and protection against cyber threats.

Industrial sensors and the edge controller communicate via RS485 and Modbus in the project. The industrial serial communication standard RS485 is reliable and built for strong noise immunity and long-distance communication. A master-slave communication architecture underpins Modbus, one of the most widely used industrial protocols. In this project, the Edge101 controller acts as the Modbus master while the sensors operate as agent devices.

For serializing sensor data, Protocol Buffers (Protobuf) were chosen for their faster, smaller binary format compared to JSON. Encoded payloads reduce bandwidth requirements and enhance embedded system communication performance. Cybersecurity was also an important part of the project because Industrial IoT systems often interact with critical infrastructure. For this reason, multiple layers of security, including AES encryption, TLS-secured MQTT communication, SQLCipher-encrypted databases, firewall filtering, VPN tunneling, and SHA256 secure logging, were implemented.

System Architecture

The sensor and edge layer, gateway layer, and server and visualization layer comprise the three main layers of the Industrial IoT architecture implemented in this project. In a daisy-chain topology, the light sensor and ultrasonic sensor are connected via RS485 Modbus communication in the sensor layer.



Kuva 1: Sensor's in daisy chain connected with Edge101

Before sending the encoded payloads over Ethernet, the Edge101 Industrial ESP32 controller continuously collects sensor values and serializes them using Protobuf. The gateway layer is responsible for decoding Protobuf messages, processing the received sensor information, storing data locally, handling monitoring operations, and implementing cybersecurity protections. Local database storage was handled by SQLite and logs are handled by SQLCipher as a secure log database, and Prometheus and Node Exporter tracked gateway system health and performance metrics. At the gateway layer, secure logging with SHA256 hashing and firewall filtering with iptables were also implemented. Processing scripts written in Python, InfluxDB, and Grafana are included in the server and visualization layer. Python scripts were used for AES decryption and MQTT data handling before storing the measurements in InfluxDB. Grafana dashboards visualized real-time sensor values and system monitoring information.

The local database is used to store the sensors data temporarily if the server gets disconnected ensuring no data is lost during communication failure. Secure logs database is used to store the status of devices and sensors. Assuming the real case scenario, if the devices get tampered or behave unexpectedly the event is recorded into secure log database and no one can modify or change except authorized individuals.

Hardware and Software Implementation

The project's edge computing device was the Edge101 Industrial ESP32 controller. Modbus communication, sensor polling, Protobuf serialization, and TCP protocol over Ethernet communication were all handled by the controller. The ultrasonic sensor used reflected ultrasonic waves to determine distance, whereas the light sensor measured environmental light intensity. RS-485 Modbus-enabled communication between the two sensors creating daisy chain. Protocol Buffers were implemented using nanopb libraries designed for embedded systems. The Protobuf workflow included sensor data collection, serialization into binary payloads, TCP over ethernet transmission to the gateway, Protobuf decoding at the gateway, and data storage in local databases. Compared to JSON, Protobuf reduced payload size and improved communication efficiency.

MQTT communication using HiveMQ with TLS encryption was implemented between the gateway and the server. Local storage used SQLite. A separate device (PC) was used as a server, where all AES encrypted data 16 bytes key were transmitted to separate device through MQTT using a Python client. In python the client then decrypts the received data and stores in Influx dB using authentication token. In this project, the local PC acts as the server, hosting both the Python client and the Influx DB database for data storage and management.

Cybersecurity Implementation

Cybersecurity was a key focus of the project. To safeguard transmitted sensor data between the server and gateway systems, AES encryption was used. During data transmission, TLS-secured MQTT ensured confidentiality, integrity, and authentication. HiveMQ was configured with TLS certificates to establish secure, encrypted communication.

The gateway also implemented iptables firewall rules to filter network traffic and restrict unauthorized access. SQLCipher encrypted the local database to prevent unauthorized tampering of logs generated by devices. Additionally, to detect tampering and preserve log integrity, a secure logging mechanism, such as a blockchain, uses SHA-256 hashing. Since each log entry was linked to the previous hash value, unauthorized changes could be detected. Between the Raspberry Pi gateway and the client machine, WireGuard VPN was set up to securely expose monitoring services without directly exposing the devices to the internet. The implemented cybersecurity architecture followed the Defense-in-Depth principle by combining multiple independent security mechanisms.

Monitoring and Visualization

Because industrial environments necessitate continuous observation of both sensor data and system health, monitoring and visualization were crucial components of the implemented Industrial IoT system. Node Exporter, Prometheus, Grafana, and InfluxDB were utilized in the project's implementation of a monitoring infrastructure. Industrial sensor measurements, network activity, and gateway performance were all visible in real

time thanks to the collaboration of these tools. On the gateway device, Node Exporter was installed to provide system-level monitoring metrics. From the Linux-based gateway system, the exporter gathered CPU, memory, network statistics, and disk usage data. These metrics confirmed the system's reliability during data transmission and processing and enabled continuous monitoring of the gateway's performance. The system for collecting metrics and monitoring was Prometheus. It collected metrics from Node Exporter regularly and stored them for monitoring and analysis. Prometheus also monitored the Industrial IoT gateway's health and operational status. The system's performance could be observed from the collected monitoring data, and stable communication between devices during operation could be ensured. Grafana dashboards displayed sensor measurements and monitoring data in real time. Grafana displayed the light and ultrasonic sensor values, as well as system metrics such as CPU utilization and network activity. Users could observe sensor measurements in real time through the dashboard, which provided a clear representation of the system state.

Because monitoring data and sensors values were continuously stored in databases, historical data visualization was also possible. InfluxDB, a time-series database for storing sensor measurements and monitoring data, was implemented. Time-series databases are well-suited for Industrial IoT systems because they efficiently handle continuously changing sensor values and timestamped monitoring data. Grafana dashboards displayed the sensor values sent from the gateway to InfluxDB. On the server, Python scripts processed MQTT data, decrypted AES-encrypted data, and added sensor values to InfluxDB. In the Industrial IoT environment, the monitoring infrastructure demonstrated real-time monitoring, historical data visualization, and gateway performance tracking successfully. Industrial sensor data and gateway metrics could be securely and reliably collected, stored, and viewed by the implemented system's complete monitoring pipeline.

Validation

And

Testing

Several communication, security, monitoring, and visualization tests were used to ensure that the implemented Industrial IoT system worked properly as a whole. This comprehensive validation aims to reassure stakeholders, engineers, cybersecurity specialists, and the system's developers about its dependability and robustness. The industrial sensors' RS485 Modbus communication with the Edge101 controller was the primary focus of the initial testing stage. The edge controller polled the light sensor and ultrasonic sensor continuously using a daisy-chain topology. The tests confirmed that the sensors reliably transmitted measurement values over the RS485 network and responded appropriately to Modbus requests. The second stage of testing verified the Protobuf implementation, with sensor values encoded into binary payloads using nanopb libraries. Transmission times and payload sizes were measured during the tests, which demonstrated the system's performance by confirming that binary serialization reduced communication overhead and increased transmission efficiency. Encryption and cybersecurity mechanisms were also validated during implementation. Tests of AES encryption and decryption demonstrated that only the correct encryption keys could access sensor data. HiveMQ successfully implemented TLS-secured MQTT communication between the server and gateway, ensuring encrypted and authenticated communication. The monitoring infrastructure can be accessed securely without exposing gateway services to the public internet, as demonstrated by WireGuard VPN testing. Cybersecurity professionals can have faith in the system's security because of these measures, which demonstrate its robust security posture. Iptables rules configured on the gateway device were used to validate the firewall. The tests verified that only authorized traffic and required communication ports remained accessible while unauthorized traffic was filtered and blocked. SQLCipher database validation also confirmed that the encrypted local database remained inaccessible without the correct pragma key, demonstrating secure local data protection. Additionally, the SHA256 hashing-based secure logging mechanism was tested and found to work well. Each new log entry contained the previous hash value, forming a chain resembling a blockchain. This safeguarded the integrity of system logs and enabled the identification of unauthorized modifications. Monitoring and visualization tests confirmed that Prometheus successfully

collected system metrics from Node Exporter while Grafana dashboards displayed real-time sensor measurements and gateway monitoring information. InfluxDB successfully stored time-series sensor data and historical records. In the Industrial IoT environment, the implemented system's stable communication, dependable monitoring, successful data visualization, and robust cybersecurity protections inspire confidence in its operational stability and performance.

Results and Discussion

By implementing secure industrial communication, real-time monitoring, encrypted data transmission, dashboard visualization, and secure database storage, the Industrial IoT system successfully achieved the project's objectives. The project demonstrated the practical integration of industrial communication protocols, edge computing, cybersecurity technologies, monitoring systems, and an end-to-end Industrial Internet of Things pipeline. By reducing the size of the payload, Protobuf improved communication efficiency over JSON. System security was improved through a collection of cybersecurity technologies, including secure logging, AES encryption, TLS, VPN tunneling, SQLCipher, and firewall filtering. Integrating multiple technologies while maintaining compatibility and stable communication across system layers was one obstacle during implementation. Configuring secure communication protocols and certificate-based encryption was another obstacle. The project successfully demonstrated a secure, functional Industrial IoT architecture despite these obstacles.

Conclusion

Using edge computing, secure networking technologies, industrial communication protocols, and real-time visualization tools, this thesis demonstrated the implementation of a secure Industrial IoT monitoring system. RS485 Modbus, Protobuf serialization, MQTT, and Ethernet networking were utilized in the implemented architecture to successfully demonstrate reliable communication between industrial sensors, edge devices, gateways, and visualization servers. Multiple cybersecurity mechanisms, including AES encryption, TLS-secured MQTT communication, SQLCipher-encrypted

databases, firewall filtering, SHA256 secure logging, and WireGuard VPN tunneling, were integrated into the system to improve communication security and data protection. Monitoring and visualization were implemented using Prometheus, Node Exporter, Grafana, and InfluxDB, providing real-time observation of sensor values and gateway system performance. The project successfully demonstrated practical Industrial IoT implementation by combining industrial networking, monitoring infrastructure, secure communication, and data visualization within a complete IIoT pipeline. The developed system is based on real-world Industrial IoT environments where continuous monitoring, cybersecurity, and reliability are crucial. Cloud deployment, Sparkplug B integration, OPC-UA support, machine learning-based anomaly detection, and redundant gateway systems for increased scalability and fault tolerance are all potential future enhancements.

Appendices

Appendix 1 – System Pipeline

The project pipeline consists of:

1. Sensors connected through RS485 Modbus
2. Edge101 controller collects data
3. Protobuf serialization performed at edge
4. Ethernet communication to gateway
5. Gateway decodes Protobuf messages
6. Data stored in SQLite and InfluxDB
7. Monitoring metrics collected by Prometheus
8. Visualization displayed in Grafana

Appendix 2 – Cybersecurity Features

Implemented cybersecurity features:

- AES encryption
- TLS-secured MQTT
- iptables firewall
- SQLCipher encrypted database
- SHA256 secure logging
- WireGuard VPN tunnel

Appendix 3 – Monitoring Stack

Monitoring stack components:

- Node Exporter
- Prometheus

- Grafana
- InfluxDB

Appendix 4 – Communication Protocols

Industrial communication technologies used:

- RS485
- Modbus
- MQTT
- TLS
- Ethernet
- Protobuf

Results

```

urbsal@bishal:~/gateway $ python tcp_socket_gateway.py
Database ready
Prometheus metrics → http://bishal:8000
TCP Gateway listening on port 5000...
Waiting for ESP32...
/home/urbsal/gateway/tcp_socket_gateway.py:57: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="Raspberry_Pi1")
Connection established between gateway and server

ESP32 Connected: ('10.0.0.2', 51261)
Expecting 8 bytes...
Raw protobuf bytes HEX :0801100118f09f01
-----
Slave ID      : 1
Sensor Type   : light_sensor
Value         : 20464
Time          : 2026-05-02 22:41:05.440717
Saved to database
Prometheus slave1_light_sensor = 20464
Expecting 6 bytes...
Raw protobuf bytes HEX :080210021815
-----
Slave ID      : 2
Sensor Type   : ultrasonic_sensor
Value         : 21
Time          : 2026-05-02 22:42:05.458436
Saved to database
Message successfully published (mid=1)

```

This section displays the scenario of protobuf encoded data and the result after decoding.

```

urbsal@bishal:~$ sudo iptables -L OUTPUT -v --line-numbers
Chain OUTPUT (policy ACCEPT 3834 packets, 330K bytes)
num  pkts bytes target     prot opt in     out     source            destination
1      0      0 ACCEPT     all  --  any    wg0      anywhere         anywhere
2      0      0 ACCEPT     tcp  --  any    any      anywhere         anywhere             tcp dpt:8883
urbsal@bishal:~$ sudo iptables -L INPUT -v --line-numbers
-bash: urbsal@bishal:~: command not found
urbsal@bishal:~$ sudo iptables -L INPUT -v --line-numbers
Chain INPUT (policy DROP 467 packets, 84426 bytes)
num  pkts bytes target     prot opt in     out     source            destination
1  2058 119K ACCEPT     all  --  lo      any      anywhere         anywhere
2   325 70585 ACCEPT     all  --  any     any      anywhere         anywhere             state RELATED,ESTABLISHED
3      0      0 ACCEPT     tcp  --  wlan0   any      anywhere         anywhere             tcp dpt:ssh
4      4    208 ACCEPT     tcp  --  any     any      anywhere         anywhere             tcp dpt:ssh
5      0      0 ACCEPT     tcp  --  eth0    any      anywhere         anywhere             tcp dpt:5000
6   318 55968 ACCEPT     udp  --  any     any      anywhere         anywhere             udp dpt:51820
7      0      0 ACCEPT     all  --  wg0     any      anywhere         anywhere
8      0      0 ACCEPT     tcp  --  wg0     any      anywhere         anywhere             tcp dpt:8000
9      0      0 ACCEPT     tcp  --  wg0     any      anywhere         anywhere             tcp dpt:9000
urbsal@bishal:~$

```

Firewall implementation using iptables

The screenshot shows the Thonny IDE interface with a SQLite database browser open. The database is named 'industrial_IOT.db' and the table 'RS485_SENSORS_DATA' is selected. The table contains 11 rows of data, including sensor IDs, slave IDs, sensor types, values, timestamps, and upload status.

	id	slave_id	sensor_type	value	timestamp	uploaded
1	4935	1	light_sensor	47000.0	2026-04-11T23:18:06.952181	0
2	4936	1	light_sensor	46000.0	2026-04-11T23:20:25.169002	0
3	4937	1	light_sensor	36000.0	2026-04-12T00:11:15.571345	0
4	4938	2	ultrasonic_sensor	4.0	2026-04-12T00:11:15.580527	0
5	4939	1	light_sensor	39000.0	2026-04-12T00:11:17.129180	0
6	4940	2	ultrasonic_sensor	4.0	2026-04-12T00:11:17.151961	0
7	4941	1	light_sensor	37000.0	2026-04-12T00:11:18.686171	0
8	4942	2	ultrasonic_sensor	4.0	2026-04-12T00:11:18.712968	0
9	4943	1	light_sensor	39000.0	2026-04-12T00:11:20.244274	0
10	4944	2	ultrasonic_sensor	4.0	2026-04-12T00:11:20.251184	0
11	4945	1	light_sensor	38000.0	2026-04-12T00:11:21.802201	0

Storing the data in local data base – Used SQLite as local database

```

● node_exporter.service
   Loaded: loaded (/etc/systemd/system/node_exporter.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-05-02 18:23:28 EEST; 3h 48min ago
 Invocation: 02feb7098159430d84b1887e5068ed23
   Main PID: 986 (node_exporter)
     Tasks: 7 (limit: 9570)
        CPU: 1min 6.847s
   CGroup: /system.slice/node_exporter.service
           └─986 /home/urbsal/node_exporter-1.7.0.linux-arm64/node_exporter

May 02 22:10:48 bishal node_exporter[986]: ts=2026-05-02T19:10:48.834Z caller=collector.go:169 level=error msg="collect>
May 02 22:10:53 bishal node_exporter[986]: ts=2026-05-02T19:10:53.828Z caller=collector.go:169 level=error msg="collect>
May 02 22:10:58 bishal node_exporter[986]: ts=2026-05-02T19:10:58.829Z caller=collector.go:169 level=error msg="collect>
May 02 22:11:03 bishal node_exporter[986]: ts=2026-05-02T19:11:03.832Z caller=collector.go:169 level=error msg="collect>
May 02 22:11:08 bishal node_exporter[986]: ts=2026-05-02T19:11:08.835Z caller=collector.go:169 level=error msg="collect>
May 02 22:11:13 bishal node_exporter[986]: ts=2026-05-02T19:11:13.829Z caller=collector.go:169 level=error msg="collect>
May 02 22:11:18 bishal node_exporter[986]: ts=2026-05-02T19:11:18.828Z caller=collector.go:169 level=error msg="collect>
May 02 22:11:23 bishal node_exporter[986]: ts=2026-05-02T19:11:23.828Z caller=collector.go:169 level=error msg="collect>
May 02 22:11:28 bishal node_exporter[986]: ts=2026-05-02T19:11:28.826Z caller=collector.go:169 level=error msg="collect>
May 02 22:11:33 bishal node_exporter[986]: ts=2026-05-02T19:11:33.826Z caller=collector.go:169 level=error msg="collect>
lines 1-20/20 (END)

```

Implementation of Node exporter to monitor the device health

```

urbsal@bishal:~$ sqlcipher /home/urbsal/gateway/secure_logs.db
SQLite version 3.46.1 2024-08-13 09:16:08 (SQLCipher 4.6.1 community)
Enter ".help" for usage hints.
sqlite> PRAGMA key ='helloworld';
ok
sqlite> SELECT * FROM SECURE_LOGS;
2026-05-02 23:05:25.912: ERROR CORE sqlcipher_page_cipher: hmac check failed for pgno=1
2026-05-02 23:05:25.912: ERROR CORE sqlite3Codec: error decrypting page 1 data: 1
2026-05-02 23:05:25.912: ERROR CORE sqlcipher_codec_ctx_set_error 1
Parse error: file is not a database (26)
sqlite>

```

This is the implementation of secure log database. The unauthorized user attempted to access the database with random key and the access id denied.

```

sqlite> PRAGMA key = 'TheEarthisround';
ok
sqlite> SELECT * FROM SECURE_LOGS;
1|Edge101|Connected|EDGE101 connected from ('10.0.0.2', 51261)|2026-05-02T22:41:03.802119|5dcf44600b286d2e8e7b8f52f20c2078477439f955244491519307cbc1431e0d|GENESIS|62d01922113fa80b22f5c526430266c5712bf9647e7e831eaae8083d33dd1955
2|Edge101|Connected|EDGE101 connected from ('10.0.0.2', 51415)|2026-05-02T22:54:48.776492|4ae426fd741e3a3048233ac9870d11750d6c9b327ff5ff6c2bb1edc130ffc2df|5dcf44600b286d2e8e7b8f52f20c2078477439f955244491519307cbc1431e0d|7df3379327b8d654fc5fa963ca960d2c7b0fa75a16327f2a610ba48e37d68b06
sqlite>

```

When the authorizer attempts to access the database with correct key it can see the logs of connection status, timestamp and SHA-256 hash. If anyone attempts to modify or temper log SHA-256 hash will change and tempering will be detectable.

Prometheus Alerts Graph Status Help

Targets

All scrape pools All Unhealthy Collapse All Filter by endpoint or labels Unknown Unhealthy Healthy

modbus_esp32 (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:8000/metrics	UP	instance="localhost:8000" job="modbus_esp32"	2.344s ago	2.609ms	

pi_health (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9100/metrics	UP	instance="localhost:9100" job="pi_health"	7.215s ago	33.231ms	

Implementation of Prometheus which collect the metrics from Node exporter and from python client

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\1\Desktop\influxdb_python_database> & C:/Python314/python.exe c:/Users/1/Desktop/influxdb_python_database/mqtt_subscriber.py
, update to latest version
  client = mqtt.Client(
Connecting to HiveMQ: 975de3b3a38e47d8a5fd7e8bbc3ae1a6.s1.eu.hivemq.cloud:8883
Waiting for messages, press Ctrl+C to stop
Connected to HiveMQ
Subscribed to topic: iot/sensors
Message received on topic: iot/sensors
{'iv': 'DOR8i/SicJhDn4KI', 'data': '1D1ebftd0/97jbWUhOg6cbuDsVZHMx+xEmlRfmb42UeT6uLcZuxeyRUW9+YXuE1tTgb7ZF2fgf2ry8SYvgMOv5pr21bYd/Ven3jvbJMi3SCZlRfRaaA1Bg3EoaZcMuHrzD/rwiDg4a6', 'tag': 'J9jTToobbk3u+qH2jmlqaA=='}
The encrypted data is
Decrypted data: {'slave_id': 1, 'sensor_type': 'light_sensor', 'value': 20464, 'timestamp': '2026-05-02T22:22:23.308204'}
Written to InfluxDB - slave:1 type:light_sensor value:20464

```

This shows the implementation of TLS -MQTT (Hive MQ) – The AES encrypted data and the data after AES decryption and store in server data base – Influx DB

influxdb™

M

Urbaal
Metropolia

Load Data

Data Explorer

Notebooks

Dashboards

Tasks

Alerts

Settings

Help & Support

Data Explorer

Table

CUSTOMIZE

Local

SAVE AS

Filter tables...

_field = value _measurement = sensor_data sensor_type = light_senso

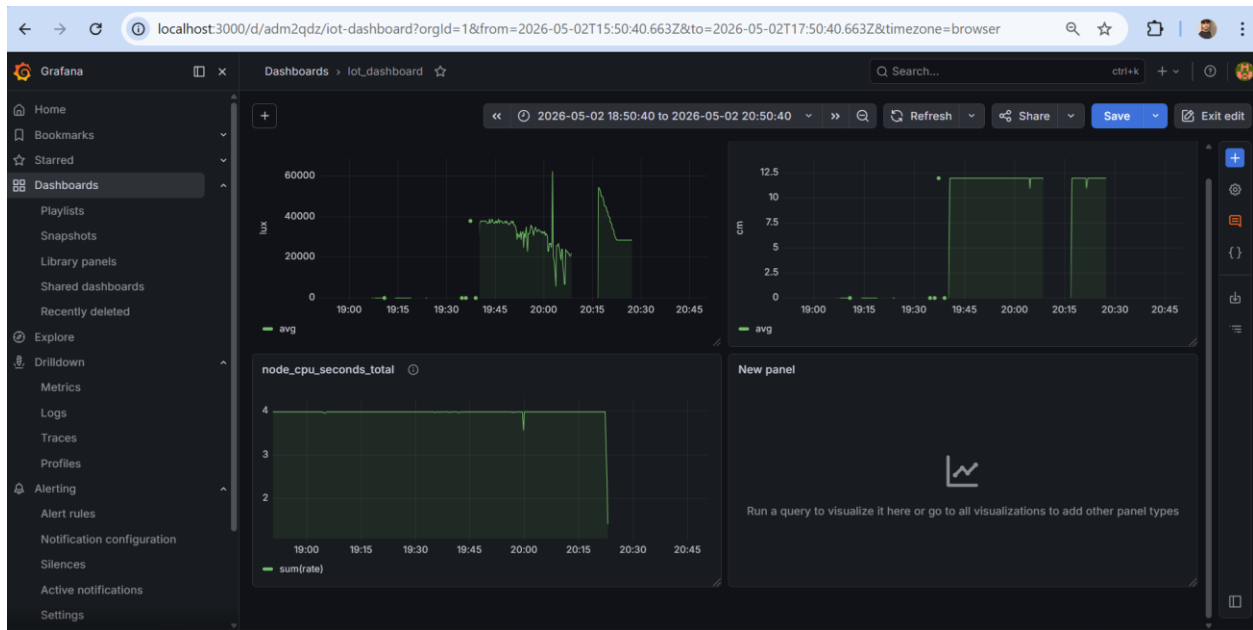
_field = value _measurement = sensor_data sensor_type = ultrasonic_

_start	_stop	_time	_value	_field	_measurement	sensor_type	slave_id	
2026-05-02 20:...	2026-05-02 21:...	2026-05-02 20:...	17	464	value	sensor_data	light_sensor	1
2026-05-02 20:...	2026-05-02 21:...	2026-05-02 20:...	18	464	value	sensor_data	light_sensor	1
2026-05-02 20:...	2026-05-02 21:...	2026-05-02 20:...	17	464	value	sensor_data	light_sensor	1
2026-05-02 20:...	2026-05-02 21:...	2026-05-02 20:...	17	464	value	sensor_data	light_sensor	1
2026-05-02 20:...	2026-05-02 21:...	2026-05-02 20:...	16	464	value	sensor_data	light_sensor	1
2026-05-02 20:...	2026-05-02 21:...	2026-05-02 20:...	16	464	value	sensor_data	light_sensor	1
2026-05-02 20:...	2026-05-02 21:...	2026-05-02 20:...	16	464	value	sensor_data	light_sensor	1
2026-05-02 20:...	2026-05-02 21:...	2026-05-02 21:...	16	464	value	sensor_data	light_sensor	1
2026-05-02 20:...	2026-05-02 21:...	2026-05-02 21:...	17	464	value	sensor_data	light_sensor	1
2026-05-02 20:...	2026-05-02 21:...	2026-05-02 21:...	17	464	value	sensor_data	light_sensor	1

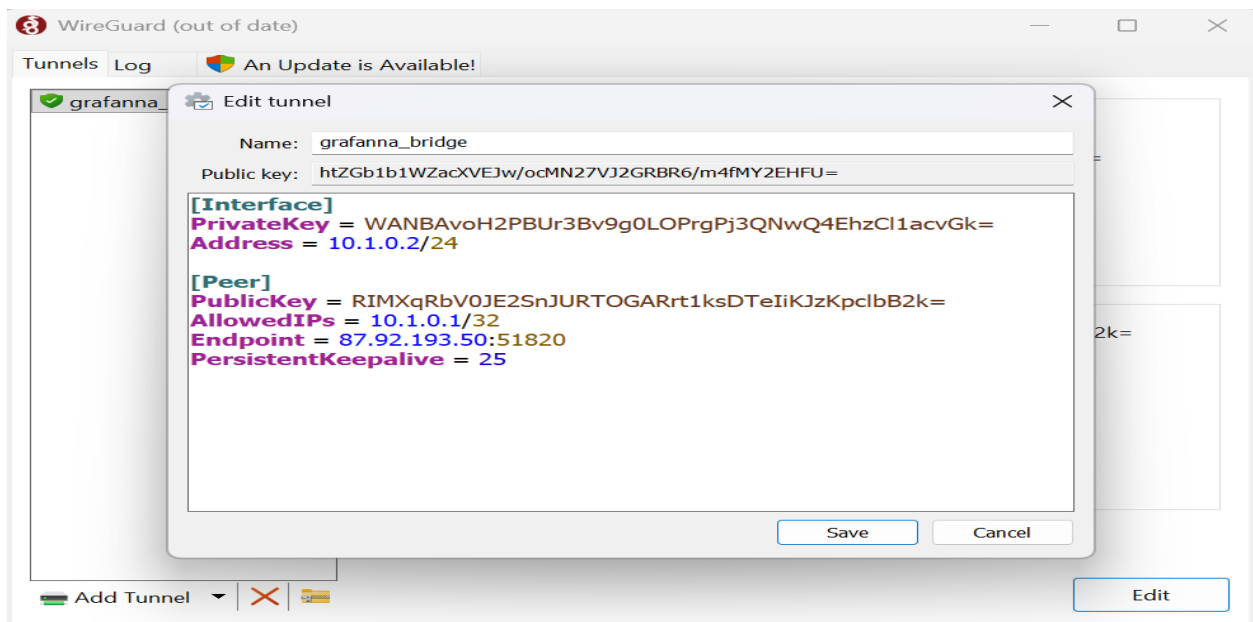
The Server database- Influx db

The screenshot shows the InfluxDB login page. It features a dark blue background with a purple gradient. In the center is a dark blue login card with the InfluxDB logo at the top. Below the logo are fields for 'Username' (containing 'Urbsal') and 'Password' (containing '*****'). A blue 'SIGN IN' button is positioned below the password field. At the bottom of the card, it displays 'InfluxDB v2.8.0', 'Server: 40a6332', and 'Frontend: 5c07739'. In the top right corner, a blue notification banner states: 'Your session has timed out. Log in again to continue.'

The server database with secure username and password



The implementation of Grafana Dashboard displaying sensor data and device health such as cup up time.



Implementation of wire guard to create a network from PC-server to gateway using ChaCha20 encryption to access Prometheus directly to integrate Grafana. This is implemented because it gives flexibility to monitor the system from anywhere with security.

